

EEL 4742C: Embedded Systems

Name:Ryan Ramdihal

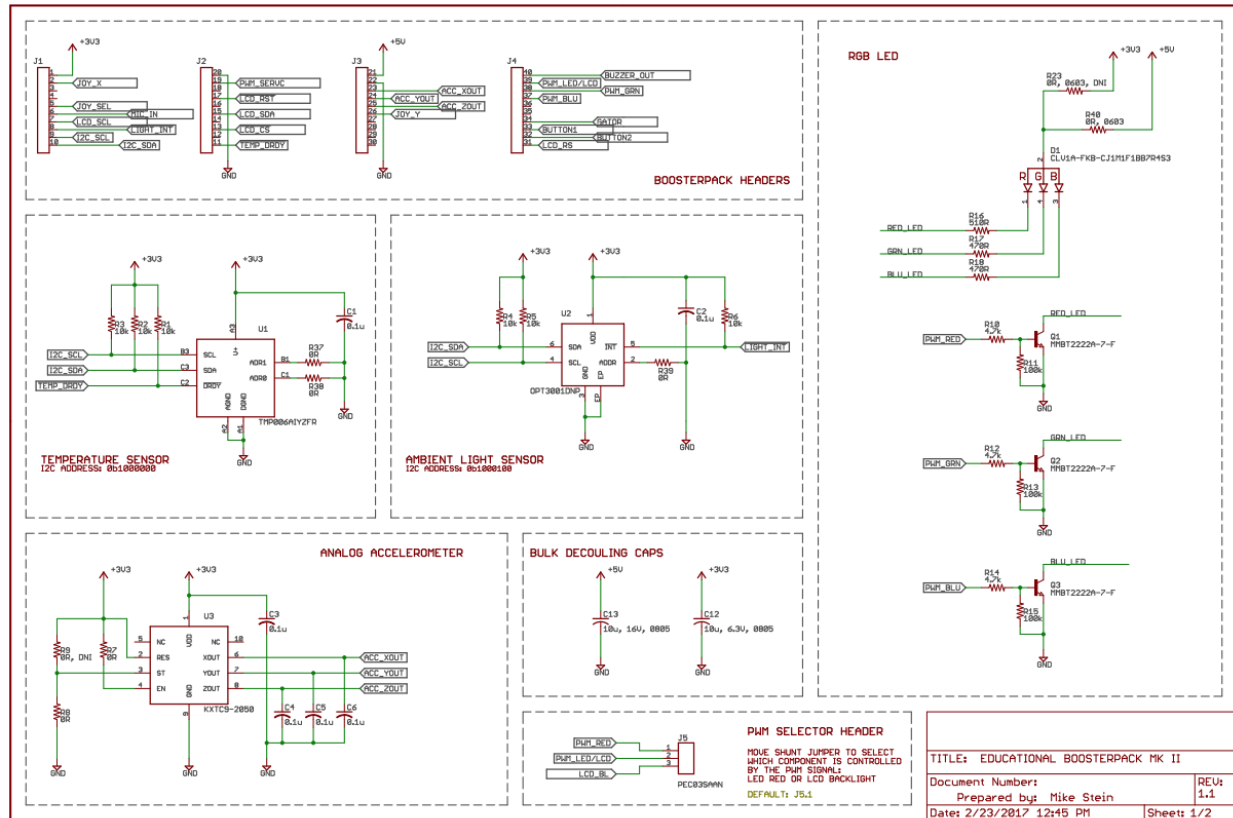
Lab 7: Inter-Integrated Circuit (I2C) Communication

Introduction:

In this lab, the objective is to interface with sensors using the TI BoosterPack MKII board, coupled with the MSP430FR6989 microcontroller. The first task involves utilizing I2C transmission to read the Manufacturer ID and Device ID registers from a sensor. This requires configuring the UART and I2C peripherals for communication and implementing functions to handle the data retrieval process. Additionally, understanding the sensor's addressing scheme and the role of pull-up resistors in I2C communication is essential. The second objective focuses on continuously monitoring data from an OPT3001 light sensor, employing a while loop to regulate the sampling frequency and transmitting the obtained data over UART for observation.

7.1: I2C Transmission

The objective of the code is to utilize I2C transmission to read the Manufacturer ID and Device ID registers from a sensor connected to the TI BoosterPack MKII board, specifically using the MSP430FR6989 microcontroller. The code initializes the UART and I2C peripherals, configuring them for communication. It then enters an infinite loop where it continuously reads the Device ID and Manufacturer ID registers from the sensor by I2C. Once retrieved, these values are transmitted over UART to the terminal for observation. The I2C functions handle the communication with the sensor, ensuring the data retrieval. The possible addresses of the sensor are dependent on the ADDR pins: 1000100 (GND), 1000101 (VDD), 1000110 (SDA), and 1000111 (SCL). The address mapped to the sensor on the BoosterPack board is 1000100. Since the I2C wires have pull-up resistors, they are active low to 10k ohms. The Device ID's address is 0x7F and the Manufacturer's ID is 0x7E. The values returned are 12289 and 21577 accordingly.



7.1 Code:

```
#include <msp430fr6989.h>

#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag
#define TXFLAG UCTXIFG // Transmit flag
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer
unsigned int ascii[10] = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57};
void Initialize_UART();
void uart_write_char(unsigned char ch);
unsigned char uart_read_char();
void uart_write_uint16(unsigned int num);
void parse_int_into_array(unsigned int num, unsigned int numSize, unsigned int* digits);
void Initialize_I2C();
int i2c_read_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int * data);
int i2c_write_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int data);
int main(void)
{
    WDTCTL = WDTPW | WDTHOLD; // stop watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
    // Configure pins to backchannel UART
    // Pins: (UCA1TXD / P3.4) (UCA1RXD / P3.5)
    // (P3SEL1=00, P3SEL0=11) (P2DIR=xx)
    P3SEL1 &= ~(BIT4|BIT5);
    P3SEL0 |= (BIT4|BIT5);
    Initialize_UART();
    // Configure pins to I2C functionality
```

```

// (UCB1SDA same as P4.0) (UCB1SCL same as P4.1)
// (P4SEL1=11, P4SEL0=00) (P4DIR=xx)
P4SEL1 |= (BIT1|BIT0);
P4SEL0 &= ~(BIT1|BIT0);
Initialize_I2C();
unsigned int manufacturerID,deviceID;
//OPT3001:
//I2C address: 0b1000100 ,page 18 of booster users guide
//Device ID = 0x7F. Manufac ID = 0x7E, page 19 of sensor data sheet
while(1){
    i2c_read_word(0b1000100, 0x7F, &deviceID); //reads the DID
    uart_write_uint16(deviceID); // writes DID to terminal
    i2c_read_word(0b1000100, 0x7E, &manufacturerID); //reads MID
    uart_write_uint16(manufacturerID); // writes MID to terminal
    delay_cycles(2000000);
    uart_write_char(10); //Write /n ascii of new line
}
}

void uart_write_uint16(unsigned int n){
    int i;
    unsigned int numSize = 1; // Initialize numSize to 1
    unsigned int num = n; // Create a copy of n to avoid modifying the original value
    // Count number of digits
    while (num /= 10) {
        numSize++;
    }
    unsigned int digits[5]; // Number will not have more than 5 digits
    // Parse the numbers digit, place into declared array
    for (i = numSize - 1; i >= 0; i--) {
        digits[i] = n % 10;
        n = n / 10;
    }
    // Write each digit over UART
    for (i = 0; i < numSize; i++) {
        uart_write_char(ascii[digits[i]]);
    }
    uart_write_char(10); // Write /n
    uart_write_char(13); // Write /r
}

void uart_write_string(char str[]){
    int i = 0;
    while(str[i] != '\0'){
        uart_write_char(str[i]);//write in array letter by letter
        i++;
    }
    uart_write_char(10); //Write /n
    uart_write_char(13); //Write /r
}

// Configure UART to the popular configuration
// 9600 baud, 8-bit data, LSB first, no parity bits, 1 stop bit
// no flow control, oversampling reception
// Clock: SMCLK @ 1 MHz (1,000,000 Hz)
void Initialize_UART(){
    // Configure pins to UART functionality

```

```

P3SEL1 &= ~(BIT4|BIT5);
P3SEL0 |= (BIT4|BIT5);
// Main configuration register
UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
// Most fields in this register, when set to zero, correspond to the
// popular configuration
UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
// Configure the clock dividers and modulators (and enable oversampling)
UCA1BRW = 6; // divider
// Modulators: UCBRF = 8 = 1000 --> UCBRF3 (bit #3)
// UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;
// Exit the reset state
UCA1CTLW0 &= ~UCSWRST;
}
// The function returns the byte; if none received, returns null character
unsigned char uart_read_char(){
    unsigned char temp;
    // Return null character (ASCII=0) if no byte was received
    if( (FLAGS & RXFLAG) == 0)
        return 0;
    // Otherwise, copy the received byte (this clears the flag) and return it
    temp = RXBUFFER;
    return temp;
}
void uart_write_char(unsigned char ch){
    // Wait for any ongoing transmission to complete
    while ( (FLAGS & TXFLAG) == 0 ) {}
    // Copy the byte to the transmit buffer
    TXBUFFER = ch; // Tx flag goes to 0 and Tx begins!
    return;
}
void config_ACLK_to_32KHz_crystal() {
    // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
    // Reroute pins to LFXIN/LFXOUT functionality
    PJSEL1 &= ~BIT4;
    PJSEL0 |= BIT4;
    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
        CSCTL5 &= ~LFXTOFFG; // Local fault flag
        SFRIFG1 &= ~OFIFG; // Global fault flag
    } while((CSCTL5 & LFXTOFFG) != 0);
    CSCTL0_H = 0; // Lock CS registers
    return;
}
void Initialize_UART_2(){
    // Configure pins to UART functionality
    P3SEL1 &= ~(BIT4|BIT5);
    P3SEL0 |= (BIT4|BIT5);
    // Main configuration register
    UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
    UCA1CTLW0 |= UCSSEL_1; // Set clock to ACLK
    // Configure the clock dividers and modulators

```

```

UCA1BRW = 6; // divider
// UCBRS = 0xEE = 1110 1110 = (bits 1, 2, 3, 5, 6, 7)
UCA1MCTLW = 0xEE00; //upper bits 15-8 check page 779*786 of family guide
// Exit the reset state
UCA1CTLW0 &= ~UCSWRST;
}
void Initialize_I2C() {
    // Configure the MCU in Master mode
    // Configure pins to I2C functionality
    // (UCB1SDA same as P4.0) (UCB1SCL same as P4.1)
    // (P4SEL1=11, P4SEL0=00) (P4DIR=xx)
    P4SEL1 |= (BIT1|BIT0);
    P4SEL0 &= ~(BIT1|BIT0);
    // Enter reset state and set all fields in this register to zero
    UCB1CTLW0 = UCSWRST;
    // Fields that should be nonzero are changed below
    // (Master Mode: UCMST) (I2C mode: UCMODE_3) (Synchronous mode: UCSYNC)
    // (UCSSEL 1:ACLK, 2,3:SMCLK)
    UCB1CTLW0 |= UCMST | UCMODE_3 | UCSYNC | UCSSEL_3;
    // Clock frequency: SMCLK/8 = 1 MHz/8 = 125 KHz
    UCB1BRW = 8;
    // Chip Data Sheet p. 53 (Should be 400 KHz max)
    // Exit the reset mode at the end of the configuration
    UCB1CTLW0 &= ~UCSWRST;
}
// Read a word (2 bytes) from I2C (address, register)
int i2c_read_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int * data) {
    unsigned char byte1, byte2;
    // Initialize the bytes to make sure data is received every time
    byte1 = 111;
    byte2 = 111;
    // Write Frame #1
    UCB1I2CSA = i2c_address; // Set I2C address
    UCB1IFG &= ~UCTXIFG0;
    UCB1CTLW0 |= UCTR; // Master writes (R/W bit = Write)
    UCB1CTLW0 |= UCTXSTT; // Initiate the Start Signal
    while ((UCB1IFG & UCTXIFG0) == 0) {}
    UCB1TXBUF = i2c_reg; // Byte = register address
    while((UCB1CTLW0 & UCTXSTT) != 0) {}
    if(( UCB1IFG & UCNACKIFG ) != 0) return -1;
    UCB1CTLW0 &= ~UCTR; // Master reads (R/W bit = Read)
    UCB1CTLW0 |= UCTXSTT; // Initiate a repeated Start Signal
    //Read Frame #1
    while ( (UCB1IFG & UCRXIFG0) == 0) {}
    byte1 = UCB1RXBUF;
    // Read Frame #2
    while((UCB1CTLW0 & UCTXSTT) != 0) {}
    UCB1CTLW0 |= UCTXSTP; // Setup the Stop Signal
    while ( (UCB1IFG & UCRXIFG0) == 0) {}
    byte2 = UCB1RXBUF;
    while ( (UCB1CTLW0 & UCTXSTP) != 0) {}
    // Merge the two received bytes
    *data = ( byte1 << 8) | (byte2 & 0xFF) );
    return 0;
}

```

```

}
// Write a word (2 bytes) to I2C (address, register)
int i2c_write_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int data) {
    unsigned char byte1, byte2;
    byte1 = (data >> 8) & 0xFF; // MSByte
    byte2 = data & 0xFF; // LSByte
    UCB1I2CSA = i2c_address; // Set I2C address
    UCB1CTLW0 |= UCTR; // Master writes (R/W bit = Write)
    UCB1CTLW0 |= UCTXSTT; // Initiate the Start Signal
    while ((UCB1IFG & UCTXIFG0) == 0) {}
    UCB1TXBUF = i2c_reg; // Byte = register address
    while((UCB1CTLW0 & UCTXSTT) != 0) {}
    // Write Byte #1
    UCB1TXBUF = byte1;
    while ( (UCB1IFG & UCTXIFG0) == 0) {}
    // Write Byte #2
    UCB1TXBUF = byte2;
    while ( (UCB1IFG & UCTXIFG0) == 0) {}
    UCB1CTLW0 |= UCTXSTP;
    while ( (UCB1CTLW0 & UCTXSTP) != 0) {}
    return 0;
}

```

7.2: Reading Measurements from the Light Sensor

This lab aims to continuously read data from the OPT3001 light sensor, by implementing a while loop. Within this loop the program paces the iterations to approximately one second by using a delay cycle loop. This delay prevents overwhelming data transmission. The program retrieves the data from the sensor using I2C communication protocols. Once the data is received, it is transmitted over UART to the terminal. Allowing for access to monitoring the levels of lux the sensor is reading. The address of the configuration register is 0x01. The sensor is consistent with what was provided in the manual.

7.2 Code:

```

#include <msp430fr6989.h>
#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag
#define TXFLAG UCTXIFG // Transmit flag
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer
unsigned int ascii[10] = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57};
void Initialize_UART();
void uart_write_char(unsigned char ch);
unsigned char uart_read_char();
void uart_write_uint16(unsigned int num);
void parse_int_into_array(unsigned int num, unsigned int numSize, unsigned int* digits);
void Initialize_I2C();
int i2c_read_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int * data);
int i2c_write_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int data);
int main(void)
{
    WDTCTL = WDTPW | WDTHOLD; // stop watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
    // Configure pins to backchannel UART

```

```

// Pins: (UCA1TXD / P3.4) (UCA1RXD / P3.5)
// (P3SEL1=00, P3SEL0=11) (P2DIR=xx)
P3SEL1 &= ~(BIT4|BIT5);
P3SEL0 |= (BIT4|BIT5);
Initialize_UART();
// Configure pins to I2C functionality
// (UCB1SDA same as P4.0) (UCB1SCL same as P4.1)
// (P4SEL1=11, P4SEL0=00) (P4DIR=xx)
P4SEL1 |= (BIT1|BIT0);
P4SEL0 &= ~(BIT1|BIT0);
Initialize_I2C();
//Write config data to OPT3001. Device ID = 0x7F. Config Reg = 0x01.
int i=0;
unsigned int data;
while(1){
    i2c_read_word(0b1000100, 0x00, &data);//result register 0x00 page 20
    data = data * 1.28;
    uart_write_string("Counter:");
    uart_write_uint16(i);
    uart_write_string("lux:");
    uart_write_uint16(data);
    uart_write_char(10);
    delay_cycles(1000000);
    i++;
}
}

void uart_write_uint16(unsigned int n){
    int i;
    unsigned int numSize = 1; // Initialize numSize to 1
    unsigned int num = n; // Create a copy of n to avoid modifying the original value
    // Count number of digits
    while (num /= 10) {
        numSize++;
    }
    unsigned int digits[5]; // Number will not have more than 5 digits
    // Parse the numbers digit, place into declared array
    for (i = numSize - 1; i >= 0; i--) {
        digits[i] = n % 10;
        n = n / 10;
    }
    // Write each digit over UART
    for (i = 0; i < numSize; i++) {
        uart_write_char(ascii[digits[i]]);
    }
    uart_write_char(10); // Write /n
    uart_write_char(13); // Write /r
}
void uart_write_string(char str[]){
    int i = 0;
    while(str[i] != '\0'){
        uart_write_char(str[i]);//write in array letter by letter
        i++;
    }
    uart_write_char(10); //Write /n

```

```

    uart_write_char(13); //Write /r
}
// Configure UART to the popular configuration
// 9600 baud, 8-bit data, LSB first, no parity bits, 1 stop bit
// no flow control, oversampling reception
// Clock: SMCLK @ 1 MHz (1,000,000 Hz)
void Initialize_UART(){
    // Configure pins to UART functionality
    P3SEL1 &= ~(BIT4|BIT5);
    P3SEL0 |= (BIT4|BIT5);
    // Main configuration register
    UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
    // Most fields in this register, when set to zero, correspond to the
    // popular configuration
    UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
    // Configure the clock dividers and modulators (and enable oversampling)
    UCA1BRW = 6; // divider
    // Modulators: UCBRF = 8 = 1000 --> UCBRF3 (bit #3)
    // UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
    UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;
    // Exit the reset state
    UCA1CTLW0 &= ~UCSWRST;
}
// The function returns the byte; if none received, returns null character
unsigned char uart_read_char(){
    unsigned char temp;
    // Return null character (ASCII=0) if no byte was received
    if( (FLAGS & RXFLAG) == 0)
        return 0;
    // Otherwise, copy the received byte (this clears the flag) and return it
    temp = RXBUFFER;
    return temp;
}
void uart_write_char(unsigned char ch){
    // Wait for any ongoing transmission to complete
    while ( (FLAGS & TXFLAG) == 0 ) {}
    // Copy the byte to the transmit buffer
    TXBUFFER = ch; // Tx flag goes to 0 and Tx begins!
    return;
}
void config_ACLK_to_32KHz_crystal() {
    // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
    // Reroute pins to LFXIN/LFXOUT functionality
    PJSEL1 &= ~BIT4;
    PJSEL0 |= BIT4;
    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
        CSCTL5 &= ~LFXTOFFG; // Local fault flag
        SFRIFG1 &= ~OFIFG; // Global fault flag
    } while((CSCTL5 & LFXTOFFG) != 0);
    CSCTL0_H = 0; // Lock CS registers
    return;
}

```



```

void Initialize_UART_20(){
// Configure pins to UART functionality
P3SEL1 &= ~(BIT4|BIT5);
P3SEL0 |= (BIT4|BIT5);
// Main configuration register
UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
UCA1CTLW0 |= UCSSEL_1; // Set clock to ACLK
// Configure the clock dividers and modulators
UCA1BRW = 6; // divider
// UCBRS = 0xEE = 1110 1110 = (bits 1, 2, 3, 5, 6, 7)
UCA1MCTLW = 0xEE00; //upper bits 15-8 check page 779*786 of family guide
// Exit the reset state
UCA1CTLW0 &= ~UCSWRST;
}

void Initialize_I2C() {
// Configure the MCU in Master mode
// Configure pins to I2C functionality
// (UCB1SDA same as P4.0) (UCB1SCL same as P4.1)
// (P4SEL1=11, P4SEL0=00) (P4DIR=xx)
P4SEL1 |= (BIT1|BIT0);
P4SEL0 &= ~(BIT1|BIT0);
// Enter reset state and set all fields in this register to zero
UCB1CTLW0 = UCSWRST;
// Fields that should be nonzero are changed below
// (Master Mode: UCMST) (I2C mode: UCMODE_3) (Synchronous mode: UCSYNC)
// (UCSSEL 1:ACLK, 2,3:SMCLK)
UCB1CTLW0 |= UCMST | UCMODE_3 | UCSYNC | UCSSEL_3;
// Clock frequency: SMCLK/8 = 1 MHz/8 = 125 KHz
UCB1BRW = 8;
// Chip Data Sheet p. 53 (Should be 400 KHz max)
// Exit the reset mode at the end of the configuration
UCB1CTLW0 &= ~UCSWRST;
}

// Read a word (2 bytes) from I2C (address, register)
int i2c_read_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int * data) {
    unsigned char byte1, byte2;
    // Initialize the bytes to make sure data is received every time
    byte1 = 111;
    byte2 = 111;
    // Write Frame #1
    UCB1I2CSA = i2c_address; // Set I2C address
    UCB1IFG &= ~UCTXIFG0;
    UCB1CTLW0 |= UCTR; // Master writes (R/W bit = Write)
    UCB1CTLW0 |= UCTXSTT; // Initiate the Start Signal
    while ((UCB1IFG & UCTXIFG0) == 0) {}
    UCB1TXBUF = i2c_reg; // Byte = register address
    while((UCB1CTLW0 & UCTXSTT)!=0) {}
    if(( UCB1IFG & UCNACKIFG )!=0) return -1;
    UCB1CTLW0 &= ~UCTR; // Master reads (R/W bit = Read)
    UCB1CTLW0 |= UCTXSTT; // Initiate a repeated Start Signal
    //Read Frame #1
    while ( (UCB1IFG & UCRXIFG0) == 0) {}
    byte1 = UCB1RXBUF;
    // Read Frame #2

```

```

    while((UCB1CTLW0 & UCTXSTT)!=0) {}
    UCB1CTLW0 |= UCTXSTP; // Setup the Stop Signal
    while ( (UCB1IFG & UCRXIFG0) == 0) {}
    byte2 = UCB1RXBUF;
    while ( (UCB1CTLW0 & UCTXSTP) != 0) {}
    // Merge the two received bytes
    *data = ( (byte1 << 8) | (byte2 & 0xFF) );
    return 0;
}
// Write a word (2 bytes) to I2C (address, register)
int i2c_write_word(unsigned char i2c_address, unsigned char i2c_reg, unsigned int data) {
    unsigned char byte1, byte2;
    byte1 = (data >> 8) & 0xFF; // MSByte
    byte2 = data & 0xFF; // LSByte
    UCB1I2CSA = i2c_address; // Set I2C address
    UCB1CTLW0 |= UCTR; // Master writes (R/W bit = Write)
    UCB1CTLW0 |= UCTXSTT; // Initiate the Start Signal
    while ((UCB1IFG & UCTXIFG0) == 0) {}
    UCB1TXBUF = i2c_reg; // Byte = register address
    while((UCB1CTLW0 & UCTXSTT)!=0) {}
    // Write Byte #1
    UCB1TXBUF = byte1;
    while ( (UCB1IFG & UCTXIFG0) == 0) {}
    // Write Byte #2
    UCB1TXBUF = byte2;
    while ( (UCB1IFG & UCTXIFG0) == 0) {}
    UCB1CTLW0 |= UCTXSTP;
    while ( (UCB1CTLW0 & UCTXSTP) != 0) {}
    return 0;
}

```

Conclusion:

This lab has provided valuable insights in interfacing sensors with microcontrollers using I2C and UART communication protocols. By effectively reading Manufacturer ID and Device ID registers from a sensor and continuously monitoring data from an OPT3001 light sensor, demonstrating proficiency in configuring and utilizing peripherals. And researching the data in the data sheet, schematics and pull-up resistors in I2C communication has given foundation for implementing these tools in other embedded applications.

Student Q&A

1. The light sensor has an address pin that allows customizing the I2C address. How many addresses are possible? What are they and how are they configured? Look in the sensor's data sheet.

4 different addresses based on the LSBs.

VDD,GND, SDA,and SCL

2. According to the light sensor's data sheet, what should be the value of the pull-up resistors on the I2C wires? Did the BoosterPack use the same values?

10kohms. It was the same as the Booster pack.

3. What I2C clock frequency do each of the eUSCI module and the sensor support?

Up to 400Khz